

좌삼우일 최종 프로젝트

딥러닝 기반 산업 현장 안전 감시 AI 시스템

박해준
이우석
이화섭
정석준

INDEX



프로젝트 개요

프로젝트 시행 목적
설정 과제 목표



시연 영상

시연 영상 시청



프로젝트 상세

프로젝트 구조
사용 환경, 기술 및 장비
코드 내용



프로젝트 진행 과정

프로젝트 개발 역할 분담 / 개발 일정
개선점

프로젝트 개요

실행 목적 / 과제 목표

프로젝트 실행 목적

딥러닝 기반 산업 현장 안전 감시 AI 시스템

현장 작업자의 헬멧 착용 여부를 실시간으로 감지하고, 위험 구역 진입 시 알림을 제공하는 안전 모니터링 시스템.



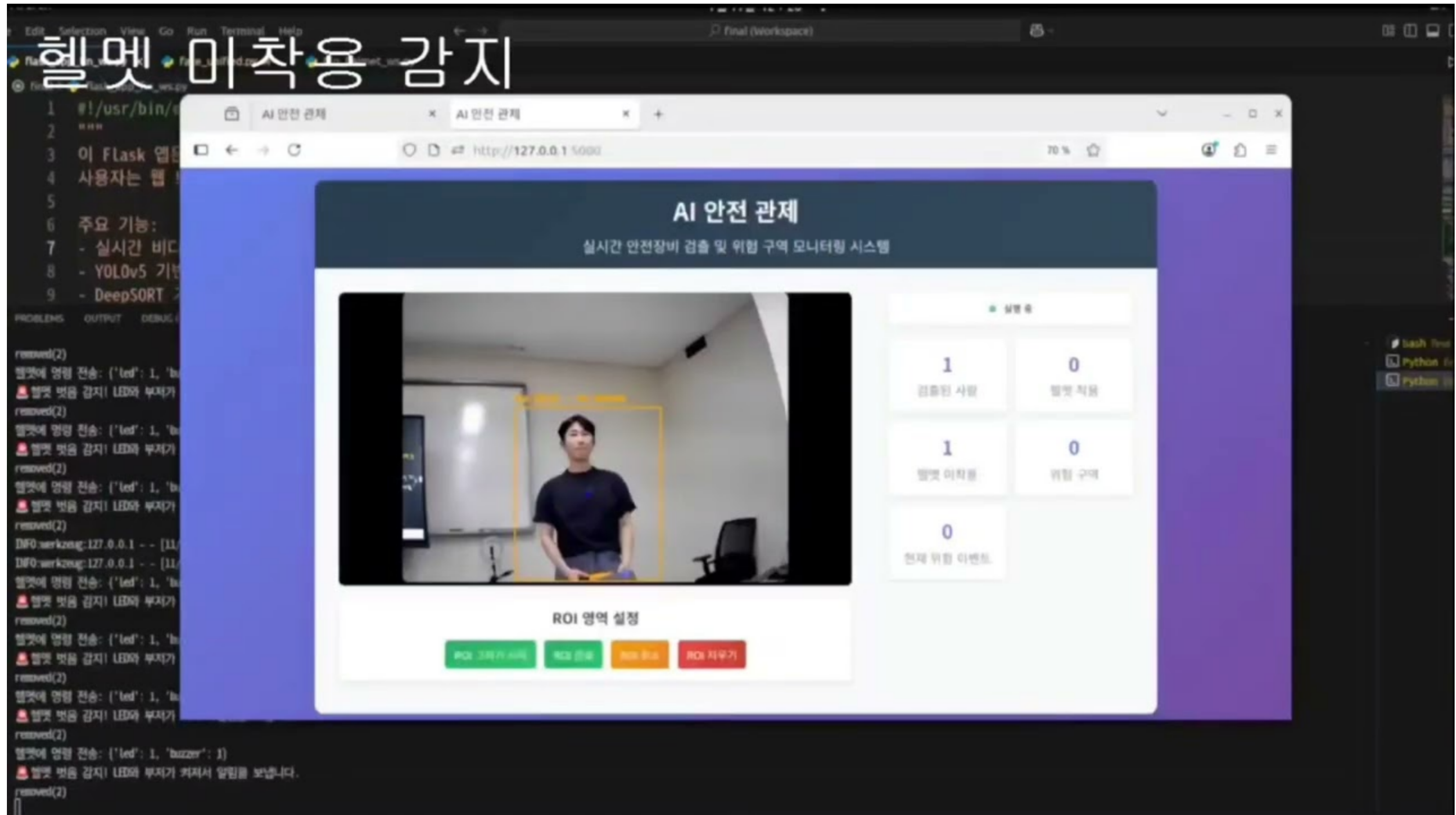
과제 목표

헬멧 착용 인식	머신러닝 모델을 이용해 헬멧 착용여부 확인
위험구역 진입 감지	ROI와 객체 탐지 모델을 이용해 위험구역에 진입하였는지 확인
얼굴 인식 기반 사용자 등록	deepface 기반의 얼굴 등록 구현 후 CCTV내에서 얼굴 인식
웹 사용 기능 구현	flask를 이용해 http서버 구축
데이터베이스구축	MYSQL을이용해 헬멧미착용 및 위험감지구역침입 데이터 저장
헬멧 자체의 알림 기능 구현	ESP8266기반으로 서버와 연결하여 헬멧 미착용 및 위험구역 진입시 작동하도록 구현

시연 영상

시연 영상

시연 영상

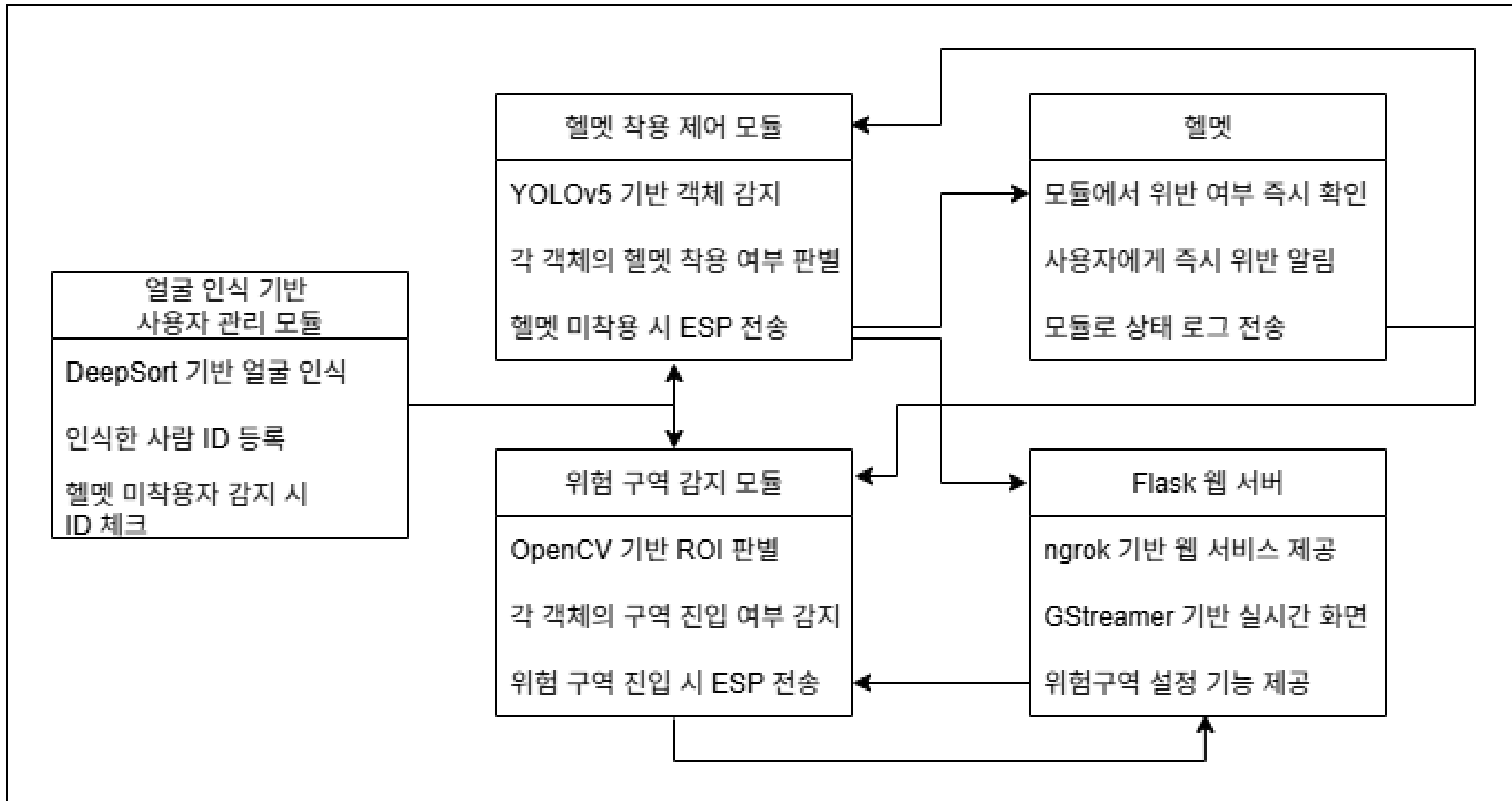


시연영상.

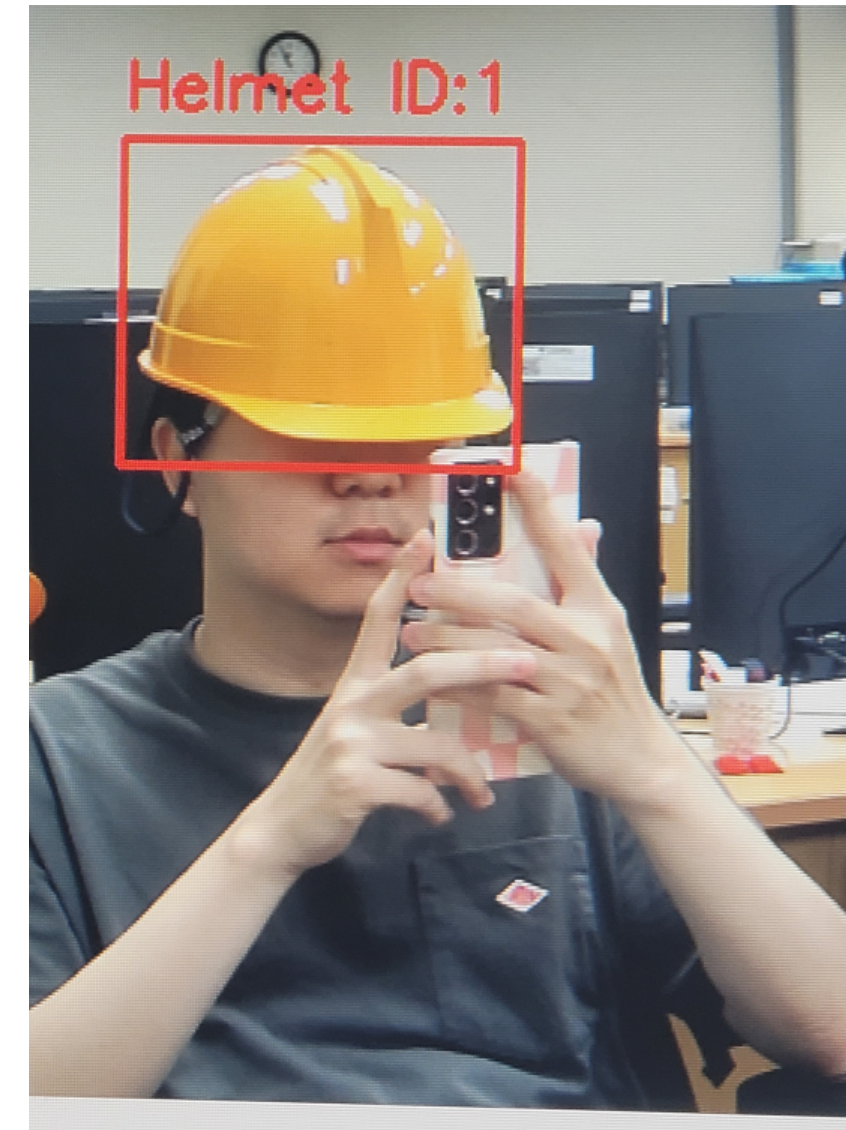
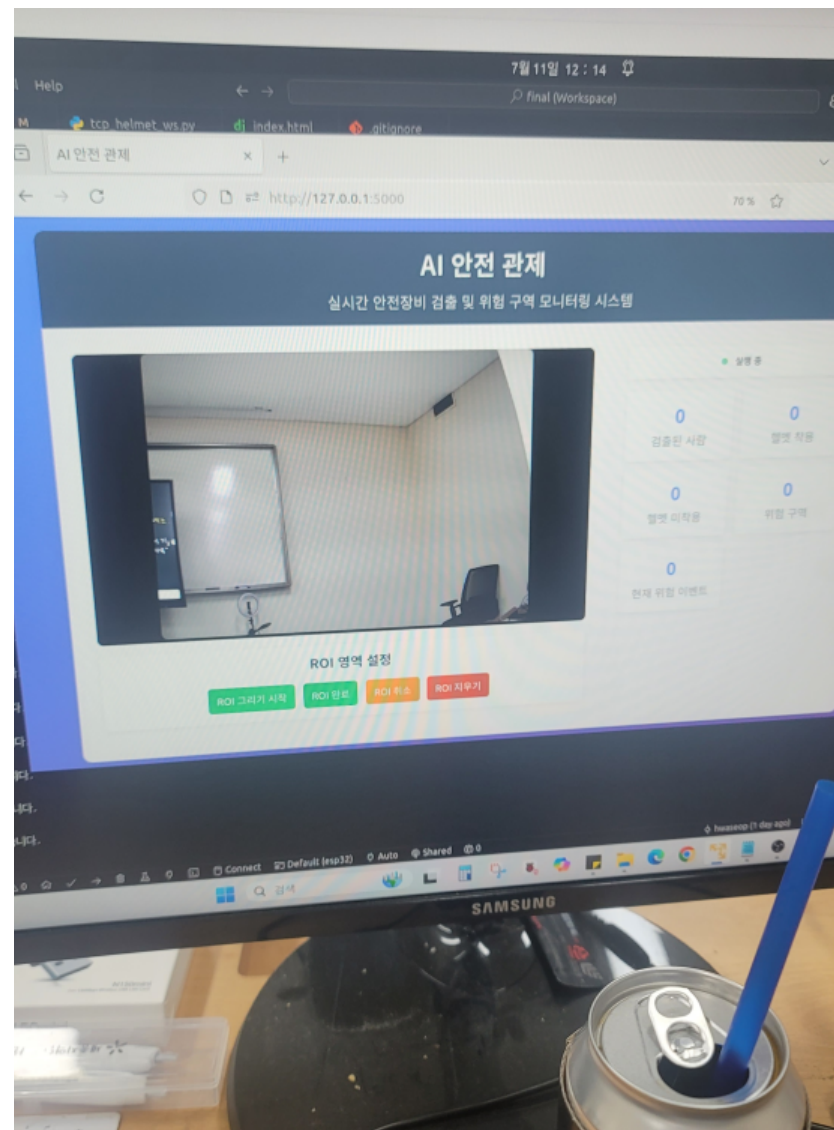
프로젝트 상세

프로젝트 구조
사용 장비 / 환경 / 기술
코드 분석

프로젝트 구조



프로젝트 구조



사용 기술

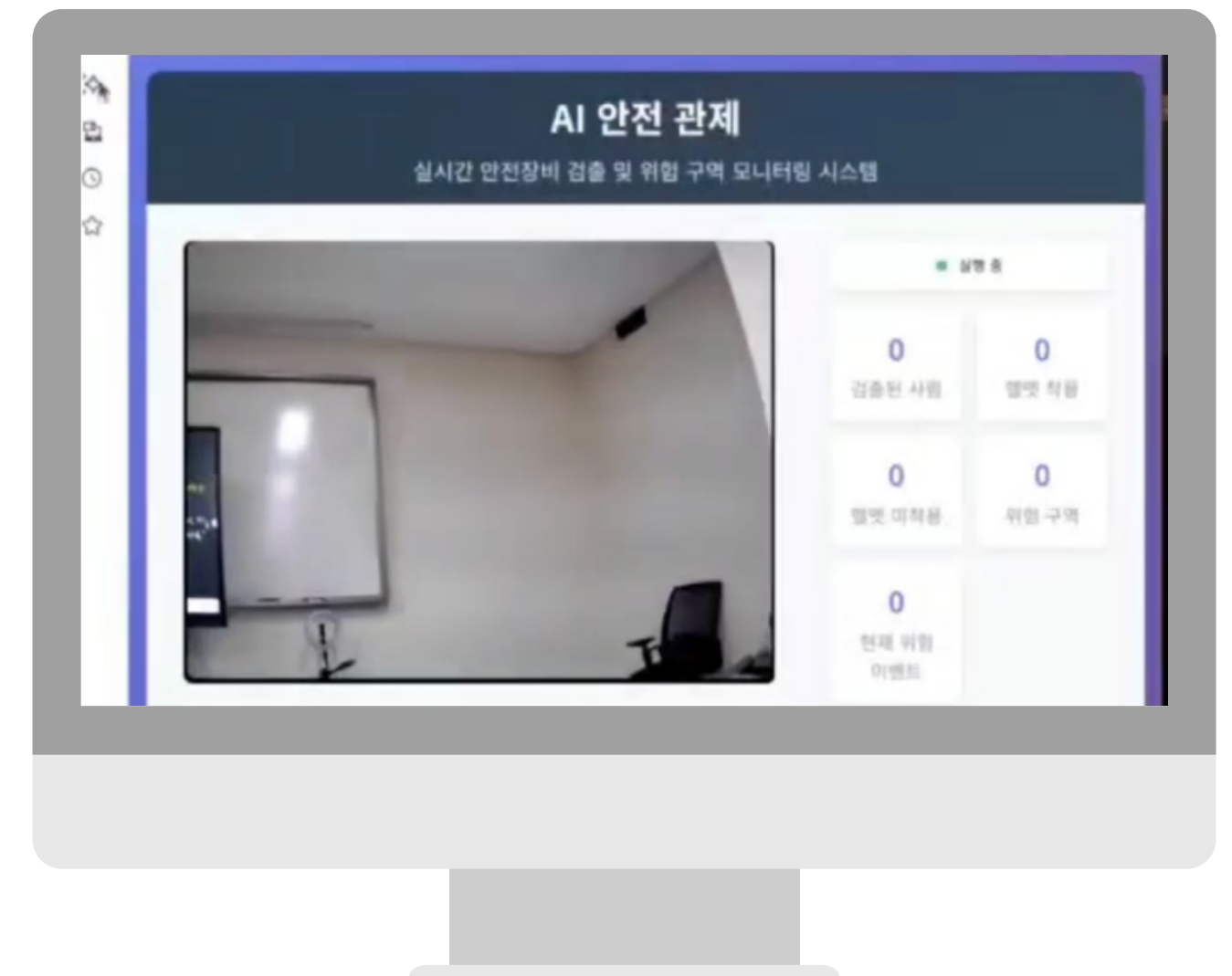
Python 기반의 라이브러리/프레임워크

Python : 통신, 딥러닝 추론, 서버, 영상처리 등 전체 프로젝트의 중심 언어

Flask : Python 기반 경량 웹 프레임워크, 실시간 스트리밍 및 API 처리에 활용

OpenCV : 이미지 처리, ROI 그리기, 영상 전처리 등 다목적으로 사용됨

Gstreamer : 오디오/비디오 스트리밍, 처리, 변환, 파이프라인 구성



／ 사용 장비

하드웨어

ESP8266 : C++ 기반의 센서 제어 및 서버 통신 기능 구현

LED / 부저 : ESP를 통해 위험구역 진입/헬멧 미착용 시 작동



코드 분석 1. 얼굴 인식

```
import os
import sys
import cv2
import numpy as np
from face_manager import FaceManager

# TensorFlow 경고 메시지 숨기기
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
os.environ['CUDA_VISIBLE_DEVICES'] = ''

# Haar Cascade 로드 (OpenCV headless에서도 작동하도록 경로 수동 지정)
cv2_base_dir = os.path.dirname(os.path.abspath(cv2.__file__))
haar_path = "/home/hwaseop/final/haarcascade_frontalface_default.xml"
face_cascade = cv2.CascadeClassifier(haar_path)

print("[DEBUG] Haar path:", haar_path)
print("[DEBUG] Exists:", os.path.exists(haar_path))

class FaceUnified:
    def __init__(self, threshold=0.5):
        self.face_manager = FaceManager() # FaceManager 인스턴스 생성
        self.cap = None
        self.threshold = threshold

    def start_camera(self):
        """웹캠 시작"""
        self.cap = cv2.VideoCapture(0)
        if not self.cap.isOpened():
            print("[X] Cannot open webcam.")
            return False
        return True

    def stop_camera(self):
        """웹캠 종료"""
        if self.cap:
            self.cap.release()
            self.cap = None
        cv2.destroyAllWindows()

    def draw_text_with_background(self, frame, text, position, font_scale=0.7,
                                text_color=(255, 255, 255), bg_color=(0, 0, 0)):
        """텍스트에 배경을 추가하여 그리기"""
        font = cv2.FONT_HERSHEY_SIMPLEX
        thickness = 2

        # 텍스트 크기 계산
        (text_w, text_h) = cv2.getTextSize(text, font, font_scale, thickness)[0]

        (x, y) = position
        x = x - text_w // 2
        y = y - text_h // 2

        # 배경 생성
        bg = np.zeros((text_h, text_w, 3), dtype=np.uint8)
        cv2.putText(bg, text, (text_w // 2, text_h // 2), font, font_scale, text_color, thickness)

        # 프레임에 배경 붙이기
        frame[y:y+text_h, x:x+text_w] = bg

    def is_db_empty(self):
        db = self.face_manager.face_db
        if not db or not isinstance(db, dict):
            return True
        for k, v in db.items():
            if isinstance(v, np.ndarray) and v.size > 0:
                return False
        return True

    def register_face_multiple_images(self, username, num_images=3):
        os.makedirs("faces", exist_ok=True)
        captured_embeddings = []

        if not self.start_camera():
            print("[X] Cannot open webcam.")
            return False

        print(f"[INFO] Registering '{username}' - capturing {num_images} face images...")
        count = 0
        while count < num_images:
            ret, frame = self.cap.read()
            if not ret:
                print("[X] Failed to read frame.")
                continue

            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            faces = face_cascade.detectMultiScale(gray, 1.3, 5)

            for (x, y, w, h) in faces:
                cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
                cv2.putText(frame, f"Face {count+1}/{num_images} - Press SPACE to capture",
                            (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
                cv2.imshow("Register Face", frame)

                key = cv2.waitKey(1) & 0xFF
                if key == 32: # SPACE
                    face_img = frame[y:y+h, x:x+w]
                    save_path = f"faces/{username}_{count+1}.jpg"
                    cv2.imwrite(save_path, face_img)

                    # 임베딩 추출
                    from deepface import DeepFace
                    emb = DeepFace.represent(img_path=save_path, model_name=self.face_manager.model_name,
                                            enforce_detection=False)[0]['embedding']
                    captured_embeddings.append(emb)
                    print(f"[OK] Captured {count+1}/{num_images} face images")
                    count += 1
```

코드 분석 2. 위험 구역 진입 / 헬멧 감지 모듈

```
# OpenCV haarcascade 경로 강제 지정
cv2_base_dir = os.path.dirname(os.path.abspath(cv2.__file__))
target_path = os.path.join(cv2_base_dir, 'data/haarcascade_frontalface_default.xml')

# 본인이 가지고 있는 haar 파일 경로
local_haar_path = "/home/hwaseop/final/haarcascade_frontalface_default.xml"

# 파일이 없으면 복사
if not os.path.exists(target_path):
    try:
        shutil.copy(local_haar_path, target_path)
        print(f"[✓] Haarcascade copied to {target_path}")
    except Exception as e:
        print(f"[X] Failed to copy haarcascade: {e}")

class FaceManager:
    def __init__(self, db_filename="face_embeddings.json", model_name="Facenet"):
        # 현재 파일 기준으로 절대 경로 생성
        base_dir = os.path.dirname(os.path.abspath(__file__))
        self.db_path = os.path.join(base_dir, db_filename) # 절대 경로로 변경
        self.model_name = model_name
        self.face_db = self.load_db()

    def load_db(self):
        if os.path.exists(self.db_path):
            try:
                with open(self.db_path, 'r') as f:
                    data = f.read().strip()
                    if not data:
                        return {}
                    loaded = json.loads(data)
                    # 리스트 + numpy 배열로 변환
                    return {k: np.array(v) for k, v in loaded.items()}
            except Exception as e:
                print(f"[X] Failed to load face database: {e}")
                return {}
```

```
class UnifiedROITracker:
    """
    A unified module for ROI-based tracking with DeepSORT.

    This module combines object detection (YOLOv5), DeepSORT tracking, and ROI-based
    danger zone detection. It can detect when tracked objects enter a defined
    region and mark them as dangerous.
    """

    def __init__(self, model_path,
                  conf_thresh=0.2,
                  max_age=30,
                  device='auto',
                  detection_interval=1,
                  threshold=0.5):
        self.threshold = threshold

    """
    Initialize the UnifiedROITracker.

    Args:
        model_path (str): Path to the YOLOv5 custom model weights
        conf_thresh (float): Confidence threshold for detections (0.0-1.0)
        max_age (int): Maximum age for track persistence in DeepSORT
        device (str): Device to run inference on ('auto', 'cuda', 'cpu')
        detection_interval (int): Run detection every N frames (1 = every frame)
    """
    # Auto-detect device if specified
    if device == 'auto':
        device = 'cuda' if torch.cuda.is_available() else 'cpu'

    try:
        # YOLOv5 커스텀 모델 로드
        self.model = torch.hub.load('ultralytics/yolov5', 'custom', path=model_path, device=device)
        self.model.conf = conf_thresh
        print(f"Model loaded successfully on {device}")
    except Exception as e:
        print(f"Error loading model: {e}")
        raise

    # DeepSORT 초기화
    self.tracker = DeepSort(max_age=max_age, n_init=3)

    from face_unified import FaceUnified # 클래스 상단에서 import
```

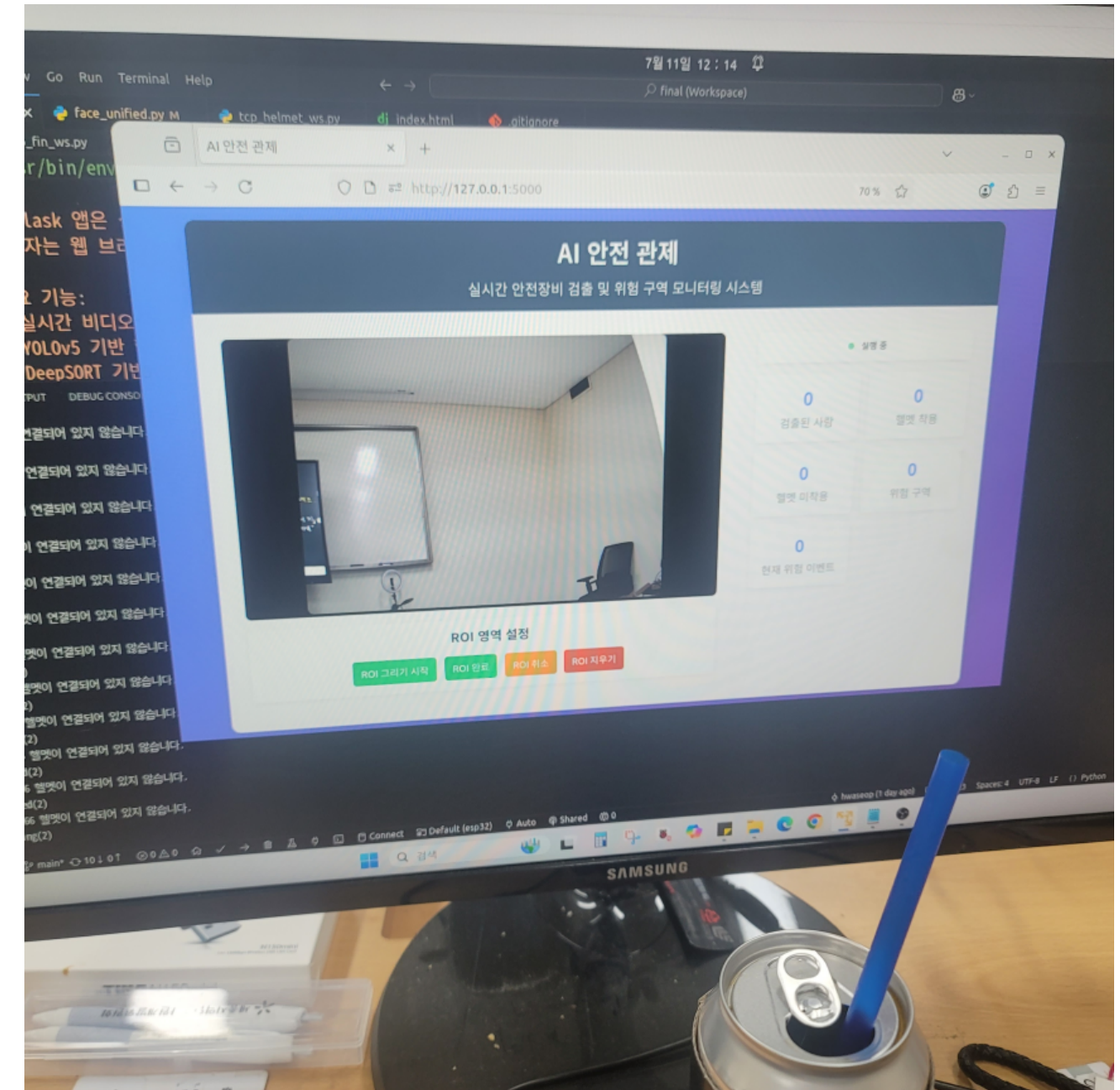
코드 분석 3. 메인 코드

```
# 기본 모듈 및 외부 의존성
from flask import Flask, render_template, Response, jsonify, request
import cv2
from functools import lru_cache
import os
import time
import threading
import subprocess
import signal
import sys
import json
from unified_roi_tracker_module import UnifiedROITracker
from database_manager_patched import DatabaseManager, init_database, get_database_manager
from face_unified import FaceUnified # FaceUnified 모듈 경로에 맞게 조정
from tcp_helmet import HelmetController # TCP 헬멧 제어 모듈 추가
face_unified = FaceUnified()

import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
# === [Flask 앱 초기화 및 전역 변수] ===
app = Flask(__name__)

# 시스템 제어 및 상태 관리용 전역 변수들
# Global variables
camera = None # OpenCV 비디오 소스
tracker = None # ROI 추적기
output_frame = None # 현재 출력 프레임
lock = threading.Lock() # 스레드 안전을 위한 잠금
stats = {} # 통계 정보
is_running = False # 트래킹 실행 중 상태
camera_thread = None # 백그라운드 영상 처리 스레드
roi_drawing_mode = False # ROI 그리기 모드
roi_points = [] # ROI 포인트
frames_processed = 0 # 처리된 프레임 수
last_frame_time = 0 # 마지막 프레임 시간

track_id_to_name = {} # 트랙 ID와 이름 매핑 (예: 헬멧 착용 여부 등)
```



코드 분석 4. 헬멧(ESP-8266)

```
FinalCpp > setup()
#include <ESP8266WiFi.h>      // ESP8266 WiFi 기능을 위한 라이브러리
#include <ArduinoJson.h>      // JSON 파싱을 위한 라이브러리

// WiFi 설정
const char* ssid = "turtle";  // WiFi SSID
const char* password = "turtlebot3"; // WiFi 비밀번호

// 서버 정보
const char* server_ip = "192.168.0.74"; // 연결할 서버의 IP 주소
const uint16_t server_port = 8000; // 연결할 서버의 포트 번호

// 하드웨어 핀 정의
#define LED_PIN D2            // LED가 연결된 핀
#define BUZZER_PIN D1         // 부저가 연결된 핀

WiFiClient client;            // TCP 통신용 클라이언트 객체

// 현재 LED와 부저 상태를 서버에 전송
void sendStatus() {
    String payload = "{\"led\":\"" + String(digitalRead(LED_PIN)) + "\",\"buzzer\":\"" + String(digitalRead(BUZZER_PIN)) + "\"}\n";
    client.print(payload);
}

void setup() {
    Serial.begin(115200);      // 시리얼 통신 시작 (디버깅용)

    pinMode(LED_PIN, OUTPUT);  // LED 핀 출력으로 설정
    pinMode(BUZZER_PIN, OUTPUT); // 부저 핀 출력으로 설정

    // 초기 상태 설정: LED 끄기, 부저 끄기 (능동 부저의 경우 HIGH가 꺼진 상태)
    digitalWrite(LED_PIN, LOW); // LED OFF
    digitalWrite(BUZZER_PIN, HIGH); // Buzzer OFF

    // 부팅 시 테스트: LED 한 번 깜빡이기
    digitalWrite(LED_PIN, HIGH);
    // digitalWrite(BUZZER_PIN, LOW);
    delay(100);
    digitalWrite(LED_PIN, LOW);
    // digitalWrite(BUZZER_PIN, HIGH);
    // delay(300);
    // digitalWrite(LED_PIN, LOW);
    // delay(300);
    // digitalWrite(LED_PIN, HIGH);

    // WiFi 연결 시도
    WiFi.begin(ssid, password);
    Serial.print("WiFi connecting");
```



프로젝트 진행 과정

프로젝트 역할분담 / 일정 / 개선점

프로젝트 역할 분담

박해준

- DB 연동(SQL)
- 학습된 모델 기반 객체 탐지 시스템 구현
- 실시간 영상 스트리밍 위한 감지 시스템 최적화

정석준

- ESP8266 센서 구현
- 서버 통신 구현(TCP)



이화섭

- 프로젝트 총괄 및 통합 담당
- 위험 감지 구역 코드 개발
- gstreamer 적용
- ngrok 기반 외부 접속 환경 구성

이우석

- YOLOv5 모델 학습
- 사용자 얼굴 등록 및 인식 코드 개발
- 테스트 통신 서버 개발

프로젝트 일정

1주차

- 아이디어 회의 및 보완 사항 추가
- 전체 시스템 흐름 및 기능 정의
- 부품 목록 및 기술 요소

2주차

- 시스템 구조 및 기능 정의 / 회의
- 위험구역(ROI) 판정 및 알림 시스템 구현
- 안전모 Training Image 분류
- YOLOv5 + DeepSORT 객체 탐지/추적
- Flask 기반 영상 스트리밍 및 최적화
- 안전모 데이터셋 분류 및 모델 학습

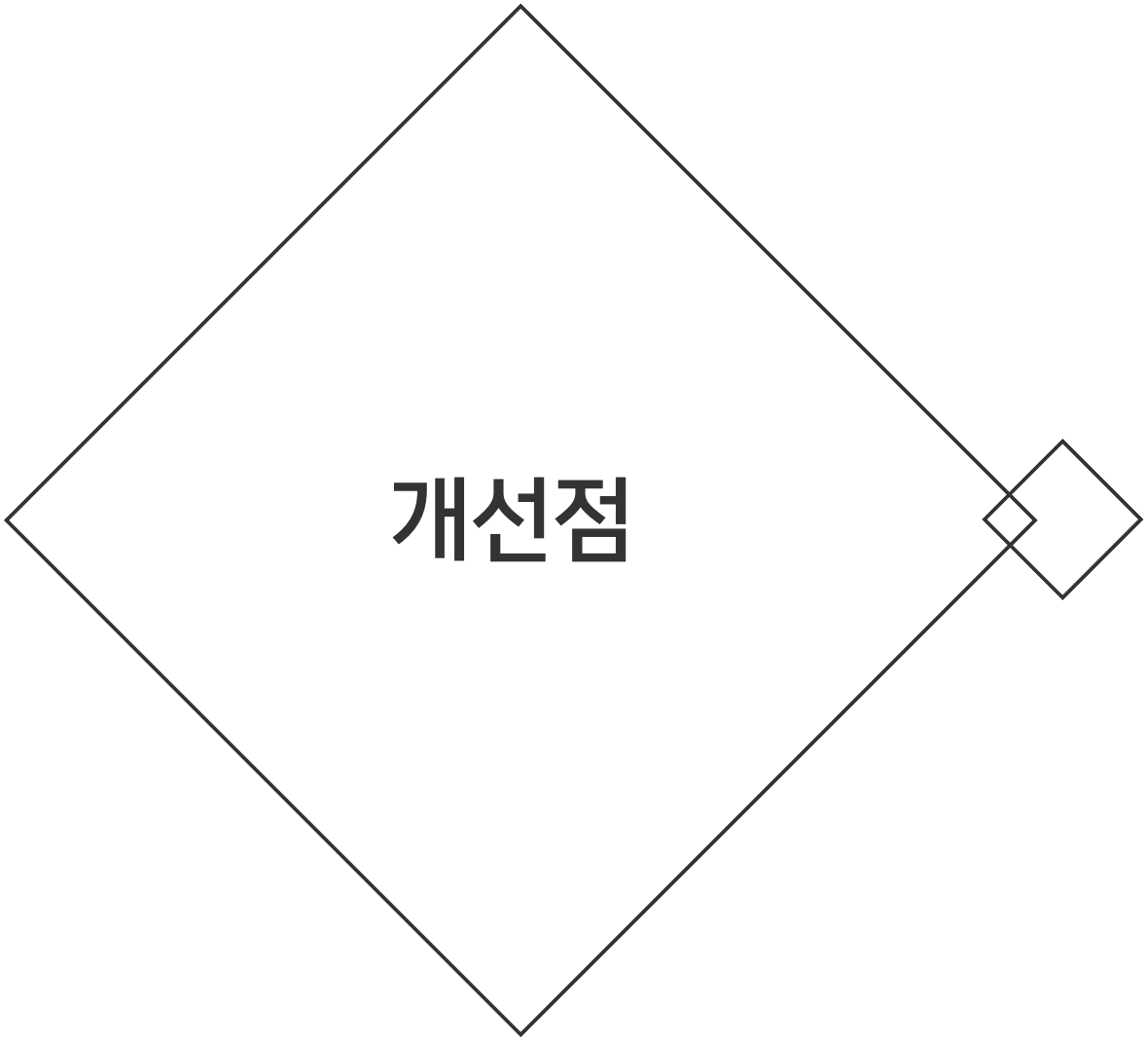
3주

- 코드 개선 및 시각화 로직 조정, 정확도 향상 방향 수립
- Flask 기반 영상 스트리밍 및 최적화
- 객체 감지 코드 개발
- 프레임 최적화
- 시스템 DB수정
- 프로젝트 코드 DB 연동

4주차

- 구현 기능 테스트 및 디버깅
- 얼굴인식 코드 리뷰, gstremlamer 설치
- 사용자 등록 시 얼굴을 3장 촬영하고 평균 임베딩을 저장하는 기능 구현
- 시스템 완성

프로젝트 개선 사항



개선점

1

프레임워크, 라이브러리 사용에 대한 경험 부족으로 최적화 미흡

2

DB의 활용도 확장 필요

3

헬멧의 기능 구현 이외에 사용자 편의성에 대한 고려 부족



Q&A